PostMortemID: Final Product Technical Report

Leny Pascal Ihirwe · BSc. Software Engineering (Machine Learning) · African Leadership University · 12 November 2026

Repository: https://github.com/leny62/postmortemid · APK: Releases page

PostMortemID tests whether the muzzle pattern of a cow, photographed alive with an ordinary smartphone, still verifies the same animal after slaughter. The product has three parts: an offline Flutter app (`mobile/`), a Python research pipeline (`ml/`) and a small upload server for the field phones (`server/`). The model is trained on a public dataset of live cattle (Xiong, Li and Erickson, 2022). The abattoir pilot starts on 13 November 2026, so this report contains no post-mortem results.

1. Scope alignment

Requirements FR1 to FR8 and NFR1 to NFR8 come from the approved proposal.

Planned feature	Intended purpose	Status	Evidence
FR1 Guided capture	Consistent framing of the muzzle and face	Completed	Square muzzle guide and tall face frame, burst camera with counter, tap to focus, 2x zoom: `mobile/lib/ui/burst_camera_screen.dart`, `camera_support.dart`
FR2 Quality check	Stop blurred, dark, bright or small photos before they reach the model	Completed	`mobile/lib/biometrics/quality.dart`; README screenshots; `test/quality_test.dart`
FR3 Research capture mode	Record study code, time point, modality, phone and time for each photo, and export the dataset	Completed	Field collection screens; slaughter time, head washing, nose condition and link certainty in the log; zip export in the analysis layout: `mobile/lib/data/field_export.dart`
FR4 Enrolment	A template from k live photos, stored with the model version	Completed	`mobile/lib/biometrics/verifier.dart`; enrol screenshot
FR5 Verification	Match, Review or No match with score and thresholds	Completed	Result screenshots; `test/workflow_test.dart`
FR6 Offline operation	Work at an abattoir with no network	Completed	TFLite model in `mobile/assets/model/`; no network call in capture, inference or storage
FR7 Records and history	Keep animals, sessions, templates and results	Completed	SQLite schema v4: `mobile/lib/data/database.dart`; History screen
FR8 Synchronisation	Move field data off the phones	Partial	Upload to the study server built (`server/`, `mobile/lib/data/field_upload.dart`); model download to phones not built
NFR1 Performance	A result within a few seconds on a low-cost phone	Partial	280 ms per photo on an emulator; low-cost phone not measured yet
NFR2 Size	Model fits a low-cost phone	Completed	14.0 MB float32 model, 50.5 MB arm64 APK
NFR3 Usability	Few taps, usable with gloves	Partial	One tap per photo in burst mode, large buttons; not tested with gloves
NFR4 Privacy	No owner data, study codes only	Partial	Ear tag never exported or uploaded (tested); it stays in the app database rather than a separate linkage file
NFR5 to NFR8	Reliability, transparency, maintainability, reproducibility	Completed	Photo saved before analysis; versions on every result; model swapped through `manifest.json`; versioned splits and seeds in `ml/data/splits/` and `ml/configs/e0.json`

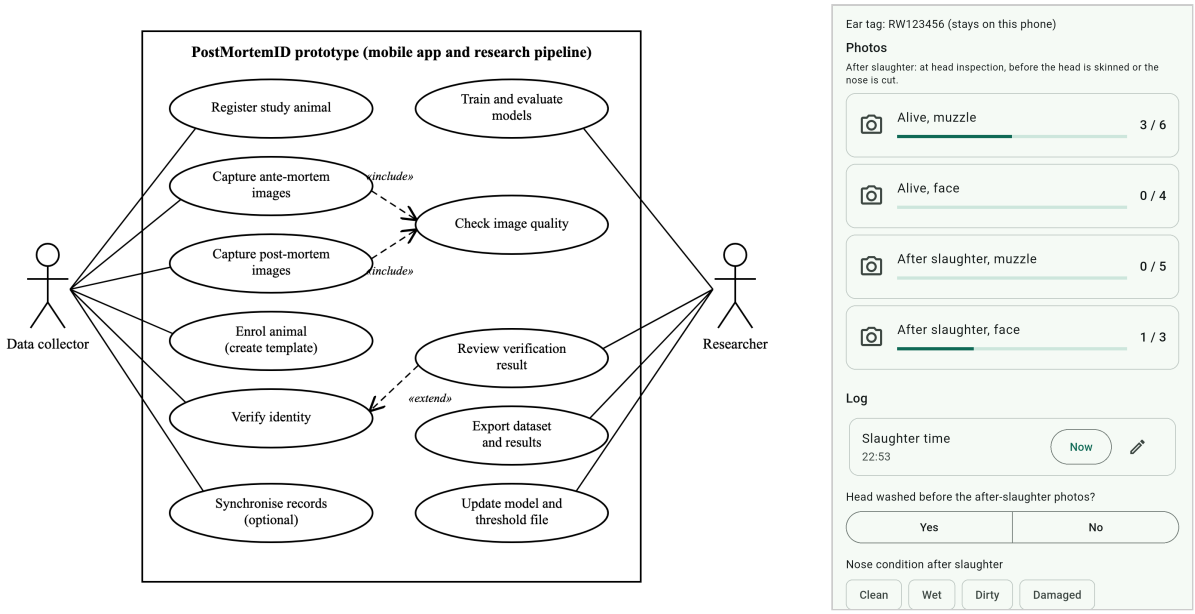


Figure 1. Use cases from the approved design (left); “Synchronise records” is now the one-way upload. Field collection screen for one animal on the Android emulator (right): photo checklist and slaughter log.

Changes from the proposal. (1) DINOv2 was replaced by frozen ImageNet MobileNetV3 features as the untrained baseline for E0, because the public set was large enough to fine-tune and DINOv2 adds a large model that cannot run on the phone. (2) The cleaned identity list of BC et al. (2026) was not available, so I wrote my own provenance screen; it removed 7 folders against their 19. (3) FR8 became a one-way upload server instead of two-way sync. Helpers at the abattoir need to send full-quality photos without WhatsApp compression, and that was the urgent part; serving models can wait until a model is trained on study data. (4) Field collection was not in the first increment; it was added because the pilot needs labelled photos from several phones.

2. Technical functionality

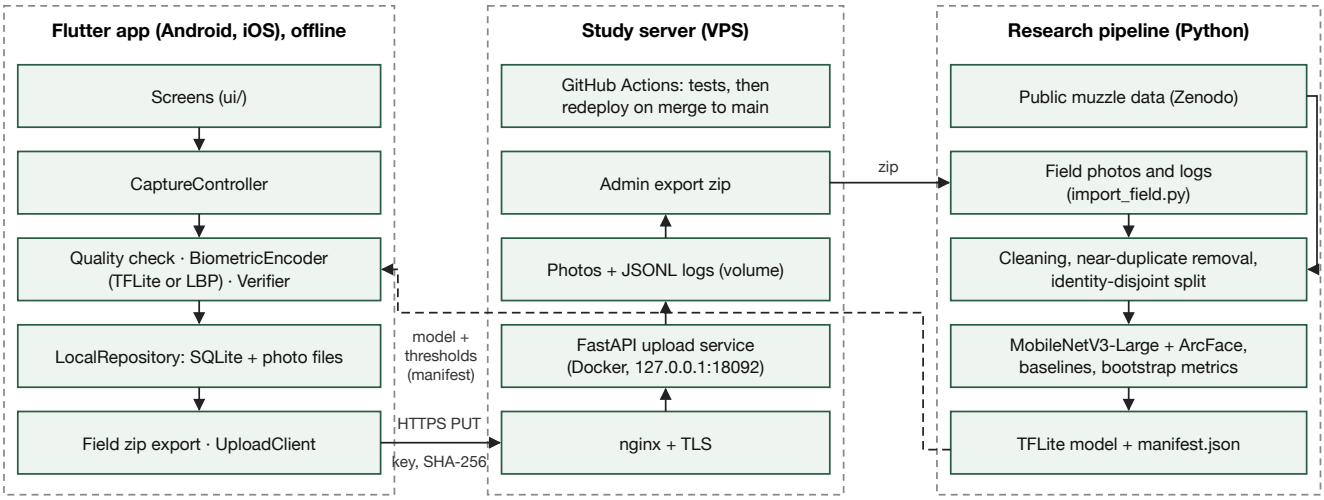


Figure 2. Architecture as built. Solid arrows are data flow; the dashed arrow is the model and thresholds shipped in the app.

App architecture. Screens (`ui/`) hold no biometric logic. `CaptureController` runs one capture session: it moves the photo into app storage, writes the database row, then runs the quality check and the encoder. `LocalRepository` is the only class that touches SQLite. `BiometricEncoder` hides the model behind an interface with two implementations, `TfliteEncoder` (MobileNetV3-Large ArcFace in LiteRT) and `LbpEncoder` (plain Dart fallback). The app reads `manifest.json` exported by the pipeline, which names the encoder, the model file, the thresholds and the quality limits.

Frontend and backend interaction. The phone talks to the server only for field upload. `UploadClient` sends `PUT /v1/photos/{animal}/{stage}/{modality}/{name}` with a per-collector bearer key, an `X-Sha256` header and the photo log as JSON. The FastAPI server checks the key, the path and the checksum, stores the file, and appends the log. The same name with the same checksum returns 200 again (a safe retry); a different checksum returns 409. Animal logs follow with `PUT /v1/animals/{id}`. An admin key downloads everything as one zip in the same layout

the app exports. The server runs in Docker on 127.0.0.1:18092 behind nginx with TLS, and a GitHub Actions workflow tests and redeploys it after each merge to `main`.

Data handling. Field photos are stored as `PM-A01/L/muzzle/<time>_<id>.jpg`, with `field_animals.csv` and `field_images.csv` beside them. `ml/scripts/import_field.py` merges zips from several phones. If two phones give different slaughter times for one animal it stops and names the conflict, and blank values never overwrite filled ones.

Algorithms.

- *Quality check:* image size, mean brightness and variance of the Laplacian for sharpness, with the same code in Python and Dart. Full-resolution photos are decoded at 1024 px width in a background isolate, so a burst does not freeze the camera.
- *Embedding and matching:* MobileNetV3-Large trained with ArcFace (additive angular margin). A template is the mean of k L2-normalised embeddings, normalised again; the score is cosine similarity.
- *Decision:* two thresholds set on development animals at 1% and 0.1% false accept. Above the strict threshold is Match, between the two is Review, below is No match.
- *Evaluation:* identity-disjoint split, near-duplicate removal with a 64-bit difference hash and Hamming distance, and 2,000 bootstrap resamples over animals in which both animals of an impostor pair are resampled.
- *Upload:* idempotent by SHA-256. The client stops at the first failure and leaves the rest queued, so a weak connection never loses or duplicates a photo.

Result on public data (E0, 52 test animals). ArcFace reached ROC-AUC 0.994, EER 2.2% [0.5, 5.6] and TAR 96.5% at the dev FAR 1% threshold, against 7.7% EER for SIFT and 11.2% for LBP. After TFLite export with the phone's own preprocessing, EER was 2.0%.

3. Testing approaches and results

On 12 November 2026, 89 automated tests passed: 46 Flutter, 39 Python (ML) and 4 server. GitHub Actions runs them, with `flutter analyze`, ruff and pyright, on every pull request.

Approach	Why I chose it	How	Result
Unit testing	The decision and quality logic must be exact	Fixed images for blur, dark, bright and small cases; fixed scores for the two thresholds; metric and split functions in pytest	All pass; each quality message fires on its case
Cross-language parity	The model was trained in Python but runs in Dart; a resize difference would shift every score	Same fixture through both pipelines (`ml/scripts/export_parity_fixture.py`, `mobile/test/parity_test.dart`)	Quality measures, LBP features and model input match
Integration testing	Upload has to survive bad networks	A local fake server in `test/field_upload_test.dart` returns 500 on the second photo, rejects a wrong key, or is switched off	Upload resumes after the 500; plain messages for a wrong key and no connection; ear tag absent from all uploaded data
On-device integration	The phone model must match the laptop	`integration_test/` runs the TFLite model on the emulator	Cosine 1.000000 against laptop output
Functional (manual)	Check the full field workflow as a collector would use it	Emulator: add animal, burst, slaughter log, export, share sheet, upload to a local server	All steps worked; two bugs found (below)
Data validation	Leakage would inflate every metric	Tests that no animal appears in two splits and that near-duplicates are removed	Pass; with a stricter duplicate cut EER rises from 2.2% to 3.9%, which I report as a limitation

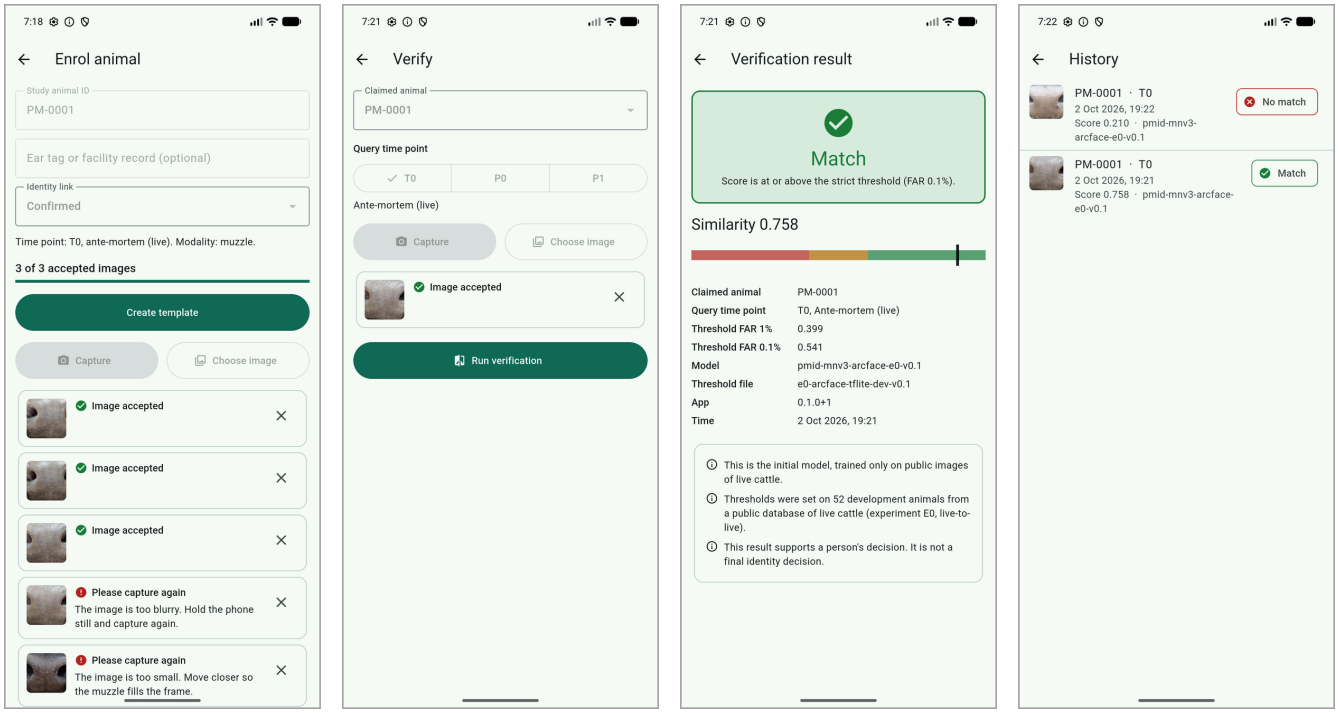


Figure 3. Different inputs on the Android emulator: enrolment with a blurred and a too-small photo rejected, verification, a Match with score, thresholds and versions, and a history holding a Match and a No match.

Improvements made after testing.

- The upload test showed the progress counter never moved: `onProgress?.call(++done, ...)` does not evaluate `++done` when no callback is given. I moved the increment out of the call.
- The manual run showed `field_animals.csv` listed demo animals with no photos, ordered by stage name. It now lists only photographed animals, ordered by capture time.
- Adding iOS showed that `getExternalStorageDirectory` is Android-only. Exports now go to the Documents folder on iOS, which is visible in the Files app.
- The server image build timed out pulling the uv base image, so the Dockerfile now installs a pinned uv with pip.

4. Performance and compatibility

Environment	What was checked	Result
Android 17 emulator (Pixel 8 Pro image), Apple M-series laptop	Inference and whole photo path	35 ms per model call; 280 ms for read, decode, quality check, resize and model on a 720 x 720 photo; 430 ms first call
iOS 26.5 simulator (iPhone 17), Xcode	Build and launch from the same Dart code	Builds and runs; model manifest loads; no camera on the simulator, so photos come from the Photos picker
Release APK, arm64	Size	50.5 MB, of which the model is 14.0 MB
Python pipeline: macOS (MPS) and Ubuntu CI (CPU)	Reproducibility	Three notebook runs on the same Mac gave identical numbers; tests pass on both
Server: Docker on Linux	Upload, summary, export	Verified against the Dart client, including non-ASCII notes

The emulator runs on a fast laptop, so these timings are not those of a low-cost phone. Timing on the two study phones is the first task of the pilot.

5. Code quality practices

- **Separation of concerns:** `ui/` (screens), `capture/` (session control and the field protocol), `biometrics/` (quality, encoders, verifier), `data/` (models, SQLite repository, CSV, export, upload).
- **Abstraction and polymorphism:** `BiometricEncoder` and `Preprocessor` are interfaces; the verifier works with either encoder, so the fast LBP encoder stands in for the model in tests.

- **Encapsulation and dependency injection:** `AppServices` passes the repository, encoder and photo check in, so tests build the real workflow on a temporary database without mocking frameworks.
- **Immutable value types:** `Animal`, `FieldLog`, `ShotSet`.
- **No monolithic scripts:** the notebook calls the `postmortemid` package; scripts are thin command-line wrappers; the server is an app factory with its own tests.
- **Consistency:** lint and type checks in CI. Comments only where the reason is not obvious. Every feature went through its own branch and pull request (7 merged PRs).

References

Xiong, Y., Li, G., & Erickson, G. E. (2022). *Beef cattle muzzle/noseprint database for individual identification* [Data set]. Zenodo. https://doi.org/10.5281/zenodo.6324361